AWS SECURITY PROJECT

Secure AWS IAM

Access Control Architecture

Implementing Least Privilege, Role-Based Access Control, Temporary Credentials, and Serverless Authentication on AWS

Least Privilege | IAM Roles | Temporary Credentials | Serverless Security

Adhithyan Sivaraman T

Cloud Security Engineer

Project Overview

Why IAM Security Matters in AWS

Security Challenges Solved

- Overly permissive IAM policies exposing resources
- Long-lived static access keys as security risk
- Lack of service-to-service authentication controls
- No enforcement of least privilege across services
- Uncontrolled cross-service access without boundaries

Project Objectives

- Enforce least privilege using IAM Users & Groups
- Implement role-based access with temporary credentials
- Secure EC2 access to S3 via IAM Instance Profiles
- Enable IMDSv2 for enhanced metadata security
- Authenticate Lambda to DynamoDB without static keys

Least Privilege

Minimal required permissions only

IAM Roles

Role-based access for services

Temp Credentials

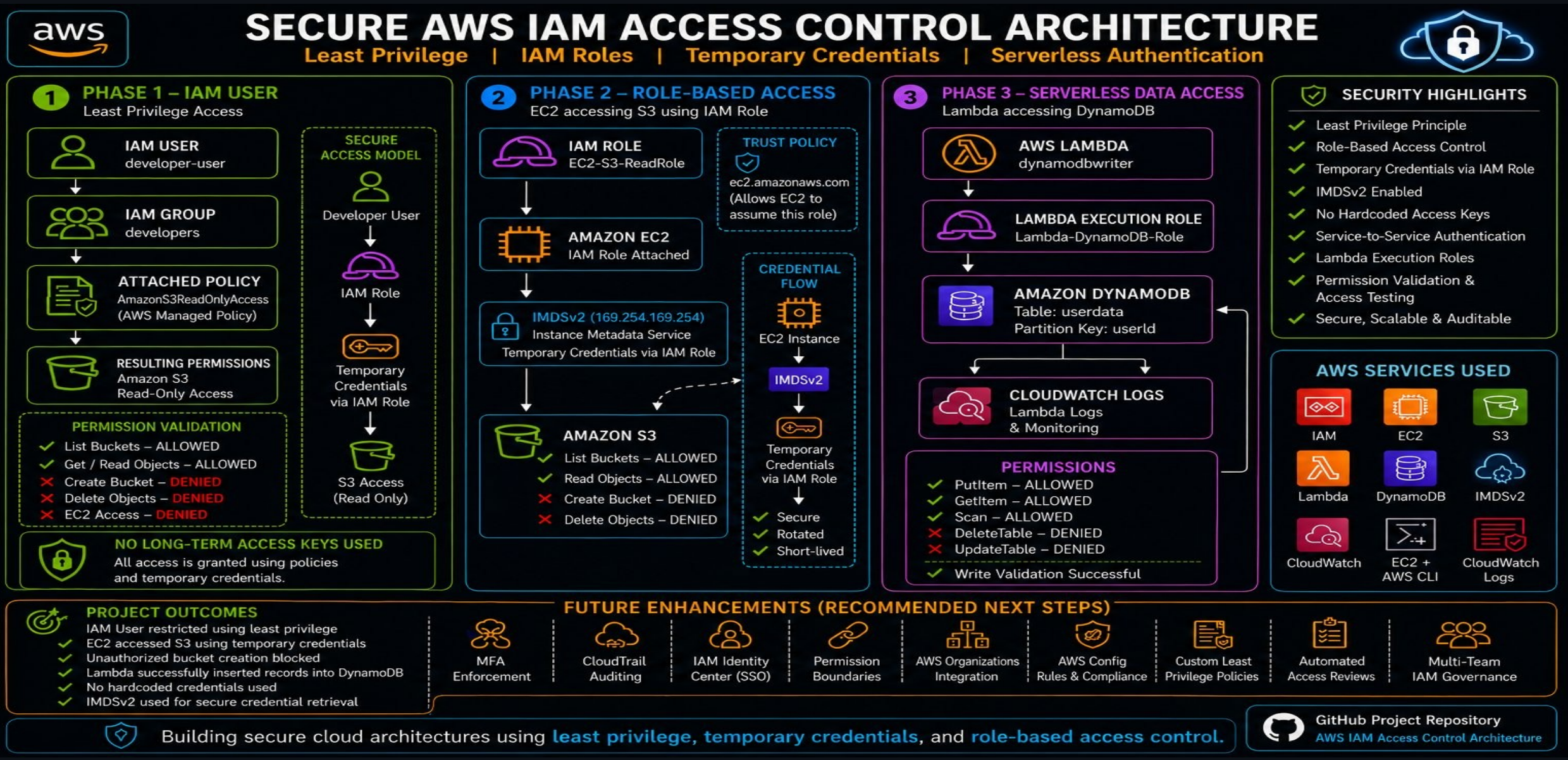
Short-lived, auto-rotated tokens

Serverless Security

Lambda with scoped execution roles

Architecture Overview

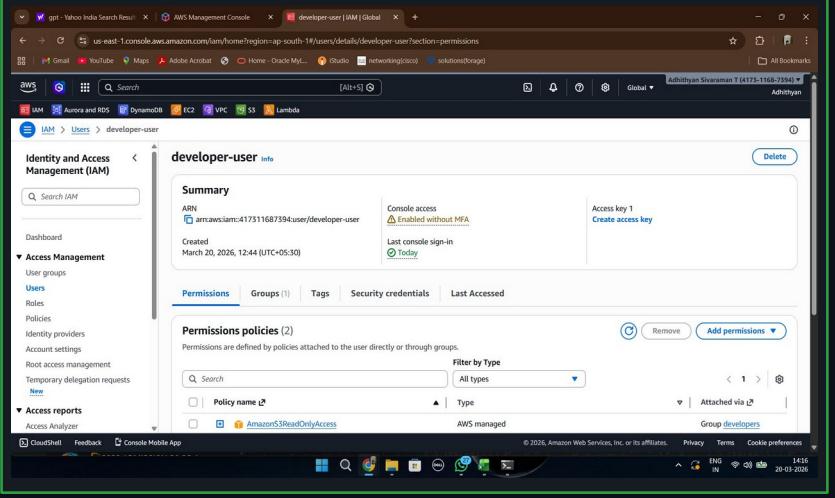
Phase 1 IAM User & Least Privilege | Phase 2 Role-Based EC2 Access | Phase 3 Serverless Lambda Auth



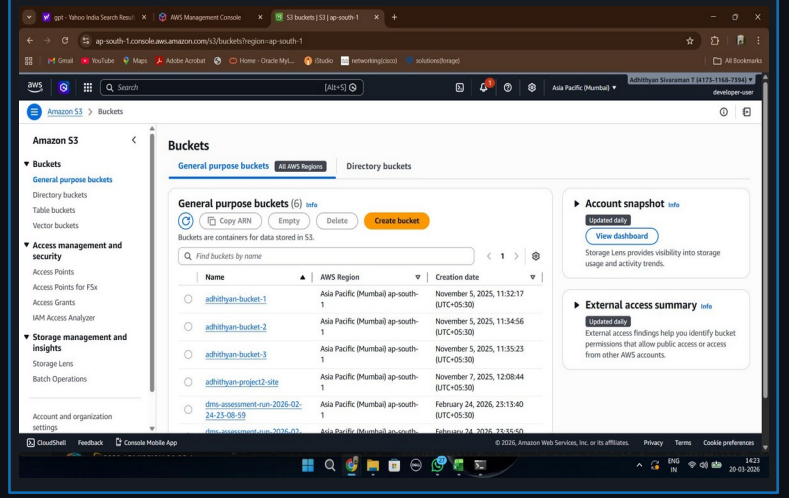
Phase 1

IAM Security & Least Privilege

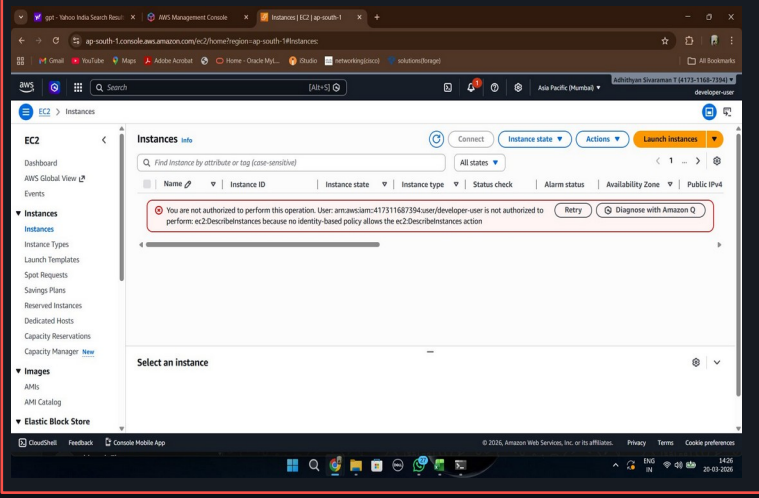
IAM User Creation



S3 Access – Success



EC2 Access – Denied



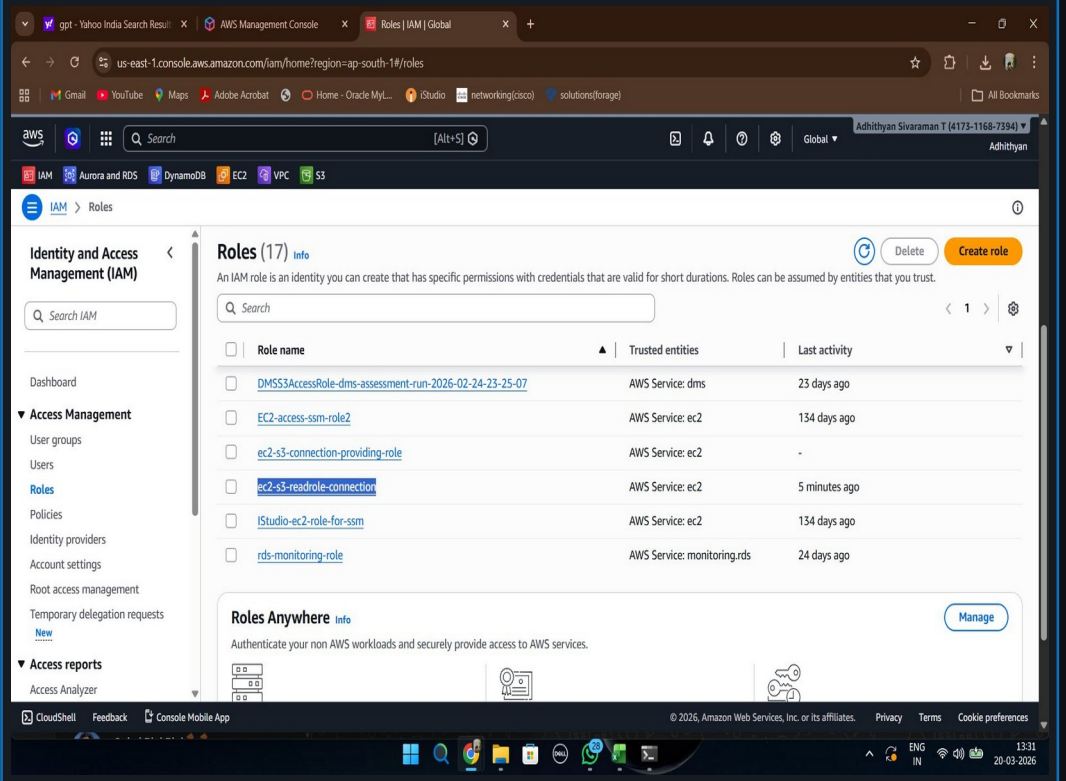
Permission Validation Matrix — Least Privilege in Action

Action	Service	Result	Reason
List Buckets	Amazon S3	✓ ALLOWED	AmazonS3ReadOnlyAccess
Get / Read Objects	Amazon S3	✓ ALLOWED	AmazonS3ReadOnlyAccess
Create Bucket	Amazon S3	✗ DENIED	ReadOnly – no write perms
Delete Objects	Amazon S3	✗ DENIED	ReadOnly – no write perms
Describe Instances	Amazon EC2	✗ DENIED	No EC2 policy attached

Phase 2

Role-Based Access Control (RBAC)

IAM Role Creation – EC2-S3-ReadRole



IAM Role

EC2-S3-ReadRole created with AmazonS3ReadOnlyAccess. No user credentials needed — role assumed by EC2 instance automatically.

Trust Policy

Trust relationship defined for ec2.amazonaws.com — only EC2 service can assume this role, enforcing strict service identity boundaries.

Instance Profile

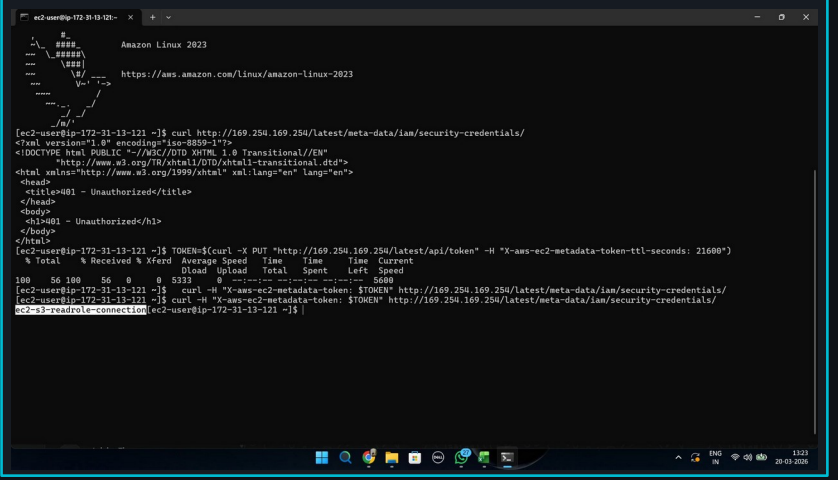
IAM Role attached to EC2 as an Instance Profile. AWS automatically manages credential rotation — no static keys required.

🔑 *Key Insight: IAM Roles eliminate the need for long-lived access keys. EC2 instances receive temporary credentials automatically via the instance profile — rotating every hour without any manual intervention.*

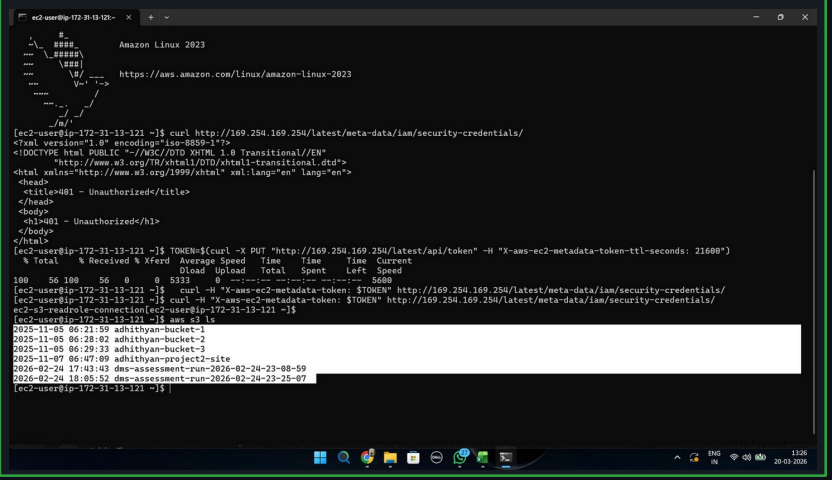
IMDSv2 & Temporary Credentials

Secure credential retrieval via Instance Metadata Service v2

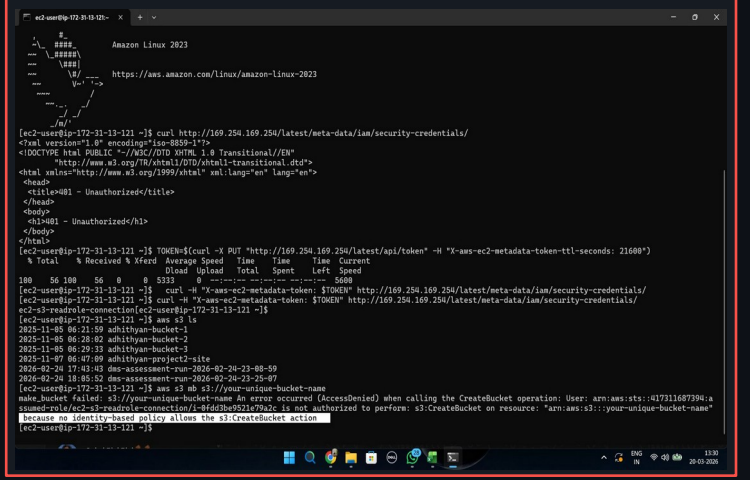
IMDSv2 Token & Role Verification



S3 Bucket List – Read Success



S3 Write – Access Denied



Security Benefits of Temporary Credentials via IMDSv2

Token Required

PUT request needed for IMDSv2 token — prevents SSRF attacks from exploiting metadata endpoint

Auto-Rotated

Credentials expire automatically — no manual rotation needed, eliminating key management risk

No Static Keys

Zero long-lived access keys on disk — compromise of instance does not expose permanent credentials

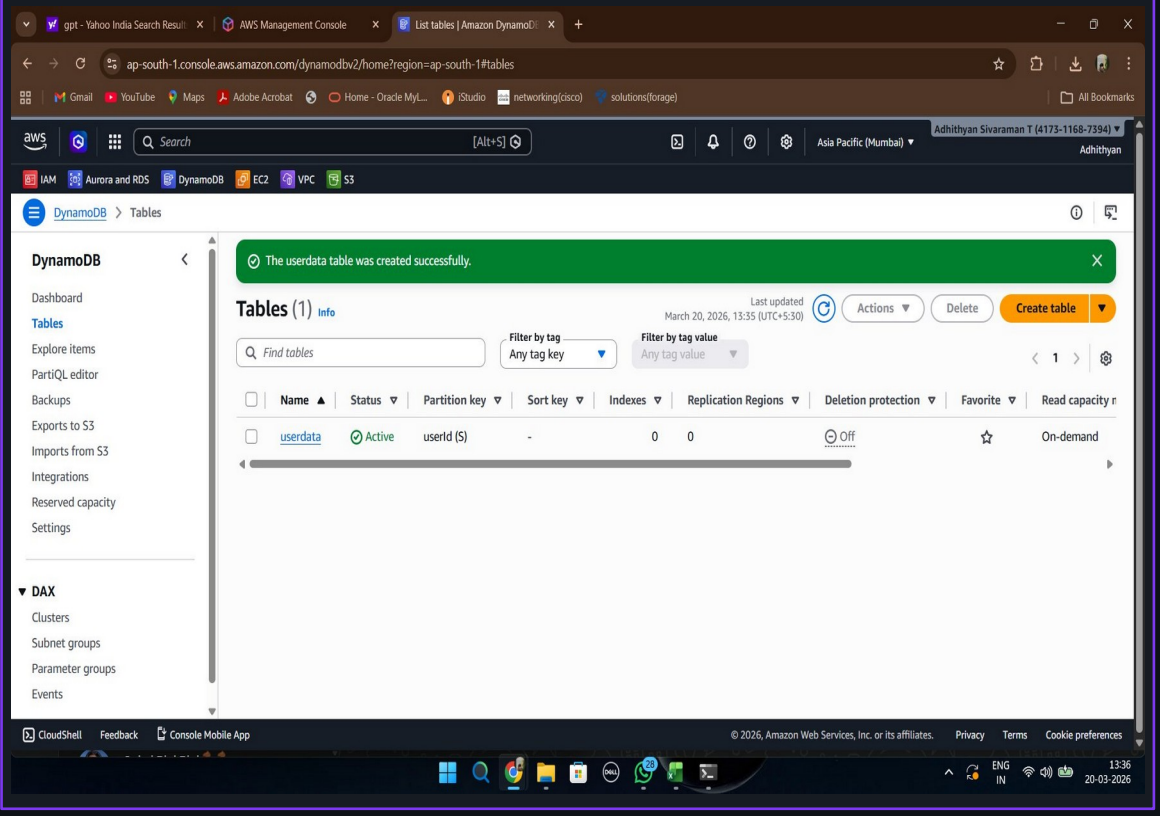
Scoped Access

Role permissions strictly scoped — read-only S3 access confirmed; write operations correctly denied

Phase 3

Serverless Access Control

DynamoDB Table Created – userdata



AWS Lambda

dynamodbwriter

Serverless function triggered by events. No server management — auto-scales based on demand.

Execution Role

Lambda-DynamoDB-Role

IAM role granting Lambda only PutItem, GetItem, Scan on the userdata table — no broader access.

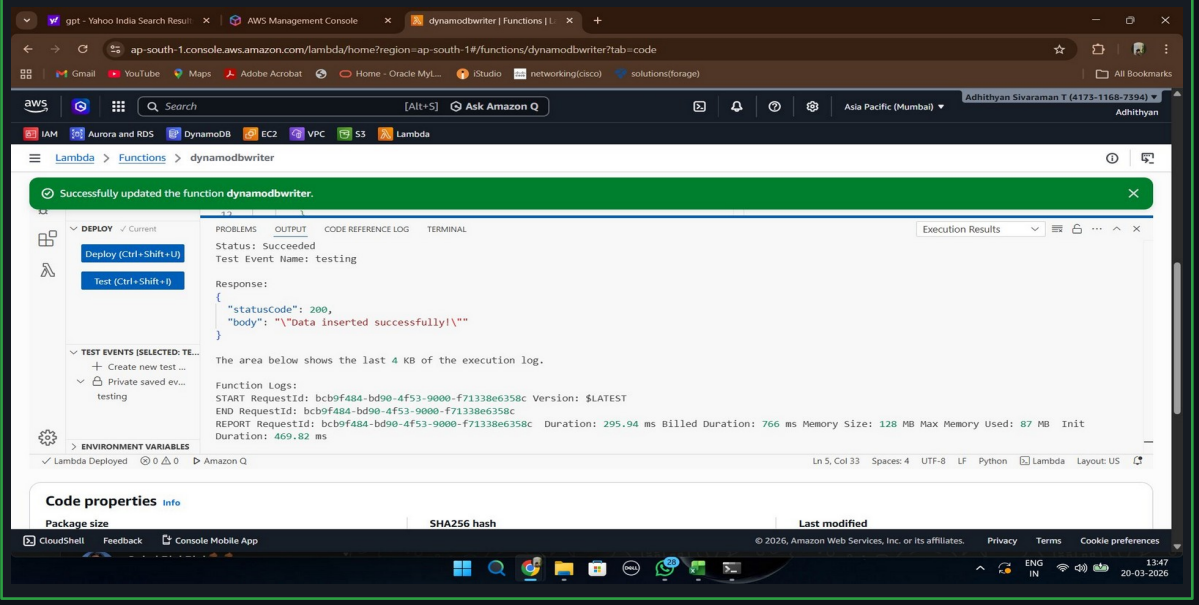
Amazon DynamoDB

Table: userdata (userId PK)

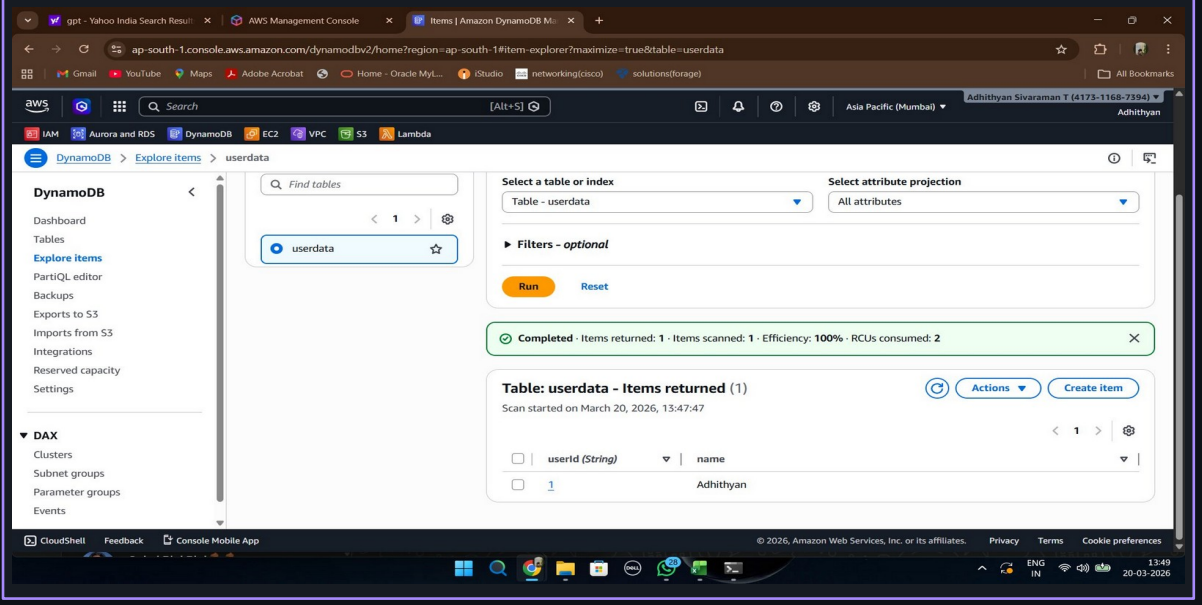
Serverless NoSQL database. Lambda authenticates via role — no connection strings or static credentials.

Lambda Validation & DynamoDB Verification

Lambda Test Execution – SUCCESS



DynamoDB Data Inserted – Confirmed



Execution

Lambda ran successfully — Status: 200, statusCode returned as expected

Data Insert

Record with userId=1, name=Adhithyan inserted into DynamoDB userdata table

CloudWatch

Full execution logs captured — duration, memory usage, request IDs all logged

RBAC Auth

Lambda authorized only via execution role — no hardcoded credentials anywhere

Security Controls Implemented

Comprehensive AWS IAM security framework across all phases

Least Privilege

Each identity receives only the minimum permissions required to perform its function — nothing more

IAM Users & Groups

developer-user assigned to developers group with AmazonS3ReadOnlyAccess managed policy

IAM Roles

EC2-S3-ReadRole and Lambda-DynamoDB-Role eliminate need for static user credentials

Trust Policies

Explicit trust relationships define which AWS services can assume each role — scoped to ec2.amazonaws.com

Temporary Credentials

All EC2 and Lambda credentials auto-expire and rotate — issued by AWS STS via IAM roles

IMDSv2 Enforced

Instance Metadata Service v2 required — token-based authentication prevents SSRF metadata attacks

No Hardcoded Creds

Zero static access keys stored in code, environment variables, or EC2 instances

Lambda Exec Roles

Lambda uses execution roles for service-to-service auth — scoped to PutItem/GetItem/Scan only

Permission Validation

All allowed and denied permissions explicitly tested and verified via AWS CLI

CloudWatch Monitoring

Full Lambda execution logs, durations, and memory metrics captured in CloudWatch

Project Outcomes

Verified results from end-to-end AWS IAM security implementation

IAM User Least Privilege Enforced

developer-user successfully restricted to S3 read-only via group policy. EC2 and all other service access correctly denied, validating least privilege implementation.

EC2 Accessed S3 via Temporary Credentials

EC2 instance assumed EC2-S3-ReadRole via instance profile. Temporary credentials retrieved through IMDSv2 — aws s3 ls executed successfully with no static keys.

Unauthorized Bucket Creation Blocked

s3:CreateBucket action correctly denied for EC2 role. AccessDenied error confirmed role is scoped to read-only — write operations are blocked as designed.

Lambda Inserted Records into DynamoDB

dynamodbwriter Lambda function executed successfully (Status: 200). Record {userId: 1, name: Adhithyan} inserted into userdata table using execution role — no credentials in code.

Zero Hardcoded Credentials

No access keys, secret keys, or passwords stored in any code, configuration, or environment variables across all three phases of the implementation.

IMDSv2 Secure Credential Retrieval

EC2 metadata service configured to require IMDSv2 token. Direct IMDSv1 access blocked (401 Unauthorized), enforcing modern secure credential retrieval protocol.

Future Enhancements

Recommended next steps for enterprise-grade IAM security maturity

MFA Enforcement

Require Multi-Factor Authentication for all IAM users, especially those with console or sensitive API access

CloudTrail Auditing

Enable AWS CloudTrail for full API audit logs — track every IAM action, role assumption, and permission change

IAM Identity Center (SSO)

Centralize workforce authentication with IAM Identity Center for single sign-on across AWS accounts

Permission Boundaries

Add permission boundary policies to cap maximum permissions for IAM entities — defense in depth for delegated access

AWS Organizations

Use Service Control Policies in AWS Organizations to enforce guardrails across all accounts in the org

AWS Config Rules

Implement continuous compliance checks — auto-detect IAM misconfigurations and policy violations in real-time

Automated Access Reviews

Schedule IAM Access Analyzer reviews to continuously audit unused permissions and overly permissive policies

Multi-Team IAM Governance

Implement team-scoped IAM boundaries with centralized policy management for large-scale engineering orgs

Key Learnings & Interview Takeaways

AWS IAM

Identity lifecycle management — users, groups, roles, policies

IAM Roles

Service identities with temporary, auto-rotated credentials

Trust Relationships

Explicit principal definitions controlling who can assume roles

IMDSv2

Token-based metadata access preventing SSRF credential theft

Temp Credentials

STS-issued creds via AssumeRole — expire automatically

Least Privilege

Grant minimum required permissions — deny by default

Lambda Exec Roles

Serverless function identity — no keys in code or env vars

DynamoDB Access

Fine-grained table-level permissions via IAM role policies

Common Interview Questions Demonstrated by This Project

Q: What's the difference between an IAM User and an IAM Role?

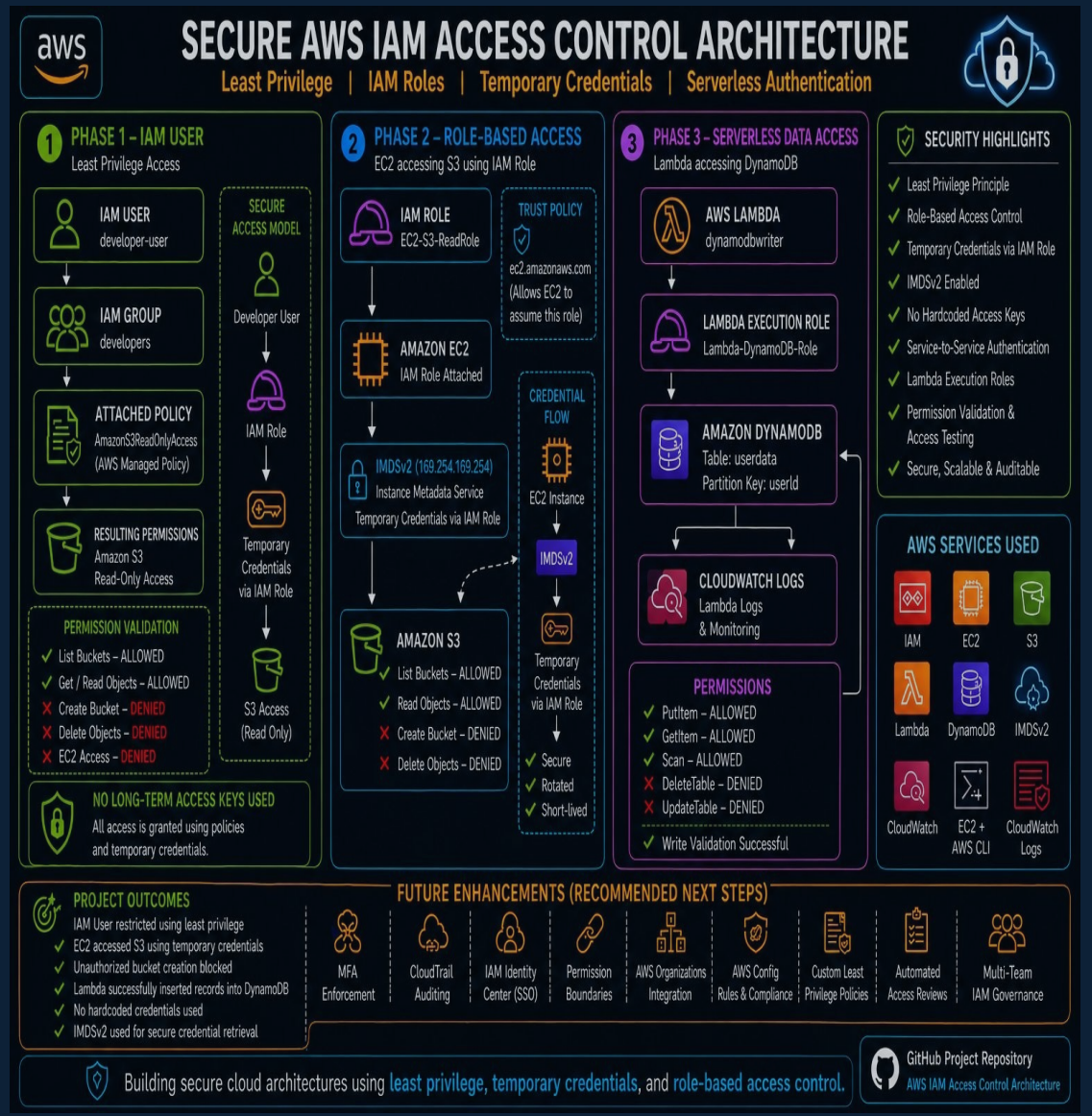
Q: How do EC2 instances securely access AWS services without access keys?

Q: Why is IMDSv2 preferred over IMDSv1 for metadata access?

Q: How does Lambda authenticate to other AWS services like DynamoDB?

Q: What is the principle of least privilege and how do you implement it?

Q: What are Trust Policies and how do they differ from permission policies?



Building Secure Cloud Architectures with AWS IAM

This project demonstrates how AWS IAM, IAM Roles, IMDSv2, Temporary Credentials, Lambda Execution Roles, and DynamoDB can be combined to implement a secure, scalable, and production-oriented cloud security architecture.

Adhithyan Sivaraman T

Thank You

AWS Cloud Security Project • 2026

IAM | EC2 | S3 | Lambda

DynamoDB | IMDSv2 | CloudWatch | IAM Roles

GitHub: github.com/Adhithyan-10/secure-aws-iam-access-control-architecture

Implemented IAM Users, Groups, Roles, IMDSv2, EC2 Instance Profiles, Lambda Execution Roles, and DynamoDB Access Controls following AWS security best practices.